pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Cash Flows and Price from Yield

WEEK 1 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

From one payment to a list of them

76.6118

Quorvane's 100, due 30 September 2031, worth today at 5.40 percent: day 1

10

coupons Quorvane also pays before then, each worth something today

A bond is a list of cash flows. Its price is the sum of their present values.

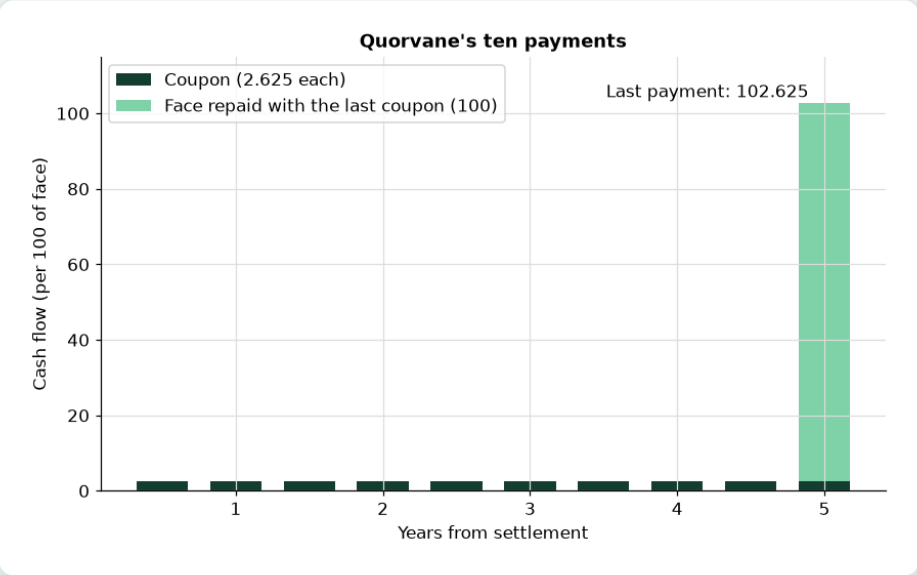
This week every bond settles on a coupon date: a whole number of half-years left, and the first coupon exactly one half-year away. Week 2 removes that condition.

Week 1: price and yield

Day 1	Time value of money
Day 2	**Cash flows and price from yield**
Day 3	The price-yield relationship
Day 4	Yield from price
Day 5	Case study: a new issue calendar

The cash flows of a bullet bond

Quorvane: 5.25 percent a year, paid twice a year, 5 years



Nine coupons of 2.625, then 102.625: the last coupon and the face. 126.25 of cash in all, per 100 of face.

```
coupon = coupon_pct / frequency
amount = coupon + 100 if k == periods else coupon
```

Every bond: payments, coupon, and total cash per 100

Issuer	Payments	Coupon each	Total cash
Pellwick Paper	6	2.3750	114.250
Quorvane Energy	10	2.6250	126.250
Dunmarrow Rail	14	3.0625	142.875
Halvering Foods	20	3.7500	175.000

The coupon per payment is the annual coupon divided by the frequency.

Price: the sum of the discounted cash flows

Pay	Cash flow	Factor	Present value
1	2.625	0.973710	2.555988
2	2.625	0.948111	2.488791
…	…	…	…
10	102.625	0.766118	78.622841
Sum			99.350327

76.6118

the redemption: day 1's number

22.7385

the ten coupons

Quorvane's price at 5.40 percent: 99.3503

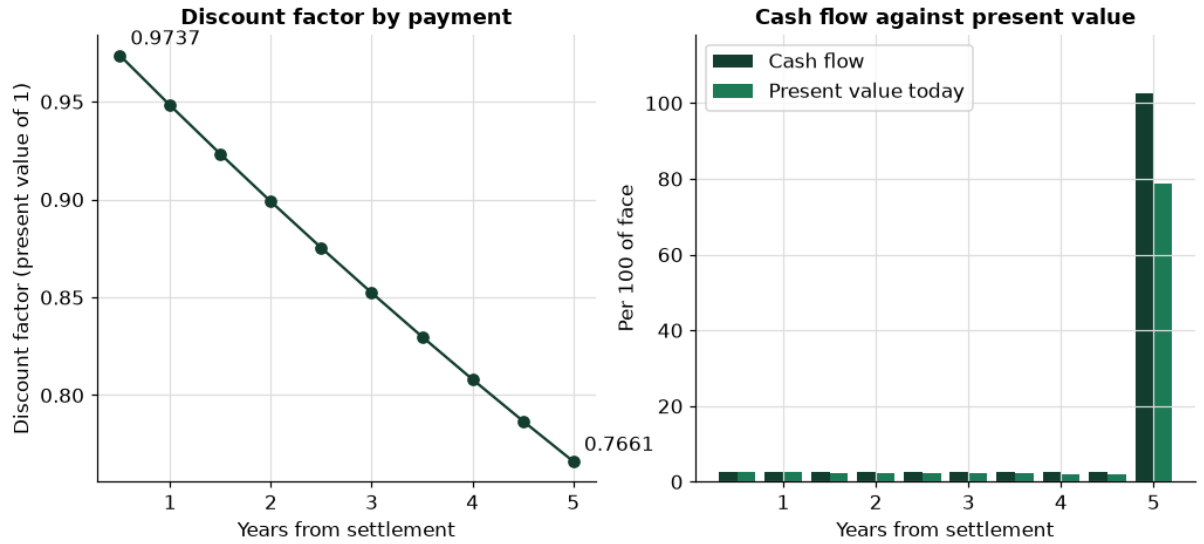
Issuer	Coupon %	Yield %	Price
Pellwick Paper	4.75	4.6	100.4159
Quorvane Energy	5.25	5.4	99.3503
Dunmarrow Rail	6.125	6.125	100.0000
Halvering Foods	7.50	7.7	98.6227

Excel side by side (tested in Excel)

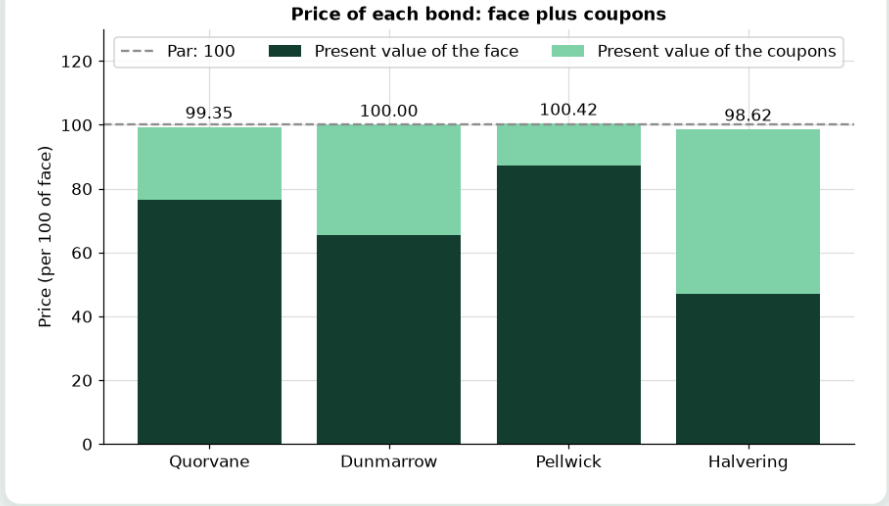
```
=NPV(5.4/100/2,B1:B10)            returns 99.350327
=SUM(C1:C10)                      returns 99.350327
=-PV(5.4/100/2,2*5,5.25/2,100)    returns 99.350327
```

B1:B10 holds the ten cash flows; C1:C10 holds =B1/(1+5.4/100/2)^A1 filled down, with 1 to 10 in A.

What the sum looks like



The last payment loses the most to waiting: 102.625 of cash is worth 78.62 today.



Same method, four shapes: a short bond is mostly face, a long bond mostly coupons.

Two functions into bondmath.py

```
def cash_flows(coupon_pct, years, frequency=2):
    """The cash flows of a bullet bond bought on a coupon date.
    ... Raises ValueError unless years x frequency is whole.
    """
    periods = years * frequency
    if periods <= 0 or abs(periods - round(periods)) > 1e-9:
        raise ValueError(...)
    coupon = coupon_pct / frequency
    ...  # (time in years, cash flow) for each payment

def price_from_yield(coupon_pct, yield_pct, years, frequency=2):
    period_yield = yield_pct / 100 / frequency
    for time_years, amount in cash_flows(coupon_pct, years, frequency):
        price += amount / (1 + period_yield) ** (time_years * frequency)
```

Shortened for the slide. The notebook shows both functions in full.

Checks that stop

a frequency other than 1, 2, or 4

years that are not whole periods:

2.3 years at frequency 2 is refused

price_from_yield calls cash_flows, so the coupon rule lives in one place.

A (time, cash flow) pair is a tuple: a fixed group of values.

Three new tests, two Excel files

Test	Checks
`test_cash_flows_count_and_final_flow`	6 percent, 5 years: 10 flows, 3.00 first, 103.00 last
`test_cash_flows_raises_on_part_periods`	2.3 years is refused
`test_price_from_yield_matches_excel`	Excel's -PV and PRICE rows

20

PV_BOND rows: Excel's -PV with a coupon, 1 to 30 years

17

PRICE rows settling on a coupon date, both day counts

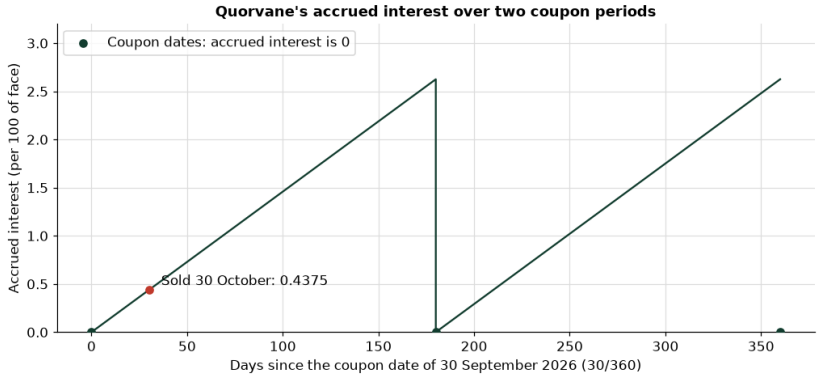
37

Excel numbers, each matched to 0.000001 per 100

6 passed: yesterday's three and today's three. In your own terminal: python -m pytest

Clean equals dirty, today

```
dirty price = clean price + accrued interest
```



On a coupon date accrued interest is 0, so clean equals dirty.

Between coupons: 99080CAE8

Item	Per 100
Clean price (PRICE)	106.726595
Accrued interest	1.983333
Dirty price	108.709928

Last coupon 102 days (30/360) before settlement. Not whole periods: today's method refuses it.

Week 2: coupon dates on day 6, accrued interest on day 7, any settlement date on day 8.

Five routes to one price

Job	Excel formula (tested)	Returns
The cash flows	`=5.25/2 in B1:B9, =5.25/2+100 in B10`	2.625, 102.625
NPV over the column	`=NPV(5.4/100/2,B1:B10)`	99.350327
Each flow, then SUM	`=B1/(1+5.4/100/2)^A1, =SUM(C1:C10)`	99.350327
One formula	`=-PV(5.4/100/2,2*5,5.25/2,100)`	99.350327

`=PRICE(DATE(2026,9,30),DATE(2031,9,30),5.25/100,5.4/100,100,2,0)` returns 99.350327

PRICE takes settlement, maturity, the coupon and the yield as decimals, redemption 100, frequency 2, and basis 0 (US 30/360). It counts the periods from the dates, so it works on any settlement date: week 2 leans on it.

On a coupon date all five agree with price_from_yield, to the twelfth decimal place.

Read the change before you save it

```
diff --git a/bondmath.py b/bondmath.py
@@ -49,3 +49,48 @@ def present_value(...)
     return amount / (1 + period_rate) ** periods
+
+
+def cash_flows(coupon_pct: float, years: float, ...
+    """The cash flows of a bullet bond bought on ...
+    ...
+def price_from_yield(coupon_pct: float, ...
```

Every new line starts with +, and no line from day 1 starts with -.

Mark	Meaning
no mark	context: unchanged
+	a line added
-	a line removed
@@	where: old and new line numbers

```
git status --short
git diff --stat
git diff bondmath.py
git log --oneline
```

Your change, a colleague's, or an AI assistant's: read the diff, then decide.

Bring it home: paste, test, read, commit

1

Your folder

Set-Location to my_bondmath, in your projects folder.

2

Paste

Today's append cells at the end of your two files, without the %%writefile line.

3

Test

python -m pytest -q: 6 passed, or 7 with day 1's exercise.

4

Read

git diff --stat and git diff: only added lines, every one with +.

5

Commit

git add, git commit, then git log --oneline.

Pasting at the end keeps anything of your own. Copy fallen behind? Take the start-of-day files from today's setup cell, and git diff shows what changed.

In Colab: download bondmath.py and test_bondmath.py from the work folder again. They replace yesterday's downloads.

The AI check

The docstring is the prompt. The code that comes back runs, and has one convention error.

1

Docstring

Units, convention, and the Excel formula: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert against Excel's reference rows. Run it: it fails.

4

Fix

Find the convention it broke, fix it, run the test again.

Today: the full annual coupon is paid every half-year: the coupon is never divided by the frequency.

Code that runs is not the same as code that is right.

What you can do now

- List the cash flows of a bullet bond: coupon / frequency, and 100 with the last
- Price it on a coupon date as the sum of its discounted cash flows
- Get the same price in Excel with NPV, a summed column, -PV, and PRICE
- Test against two Excel reference files, and say why clean equals dirty today
- Read a change with git diff before you commit it

Week 1, Day 3 The price-yield relationship: par, premium, and discount, and how far a price moves